

# Aligned E2E MC-GT-LeWM

## 方法说明与损失函数

TDWM 项目

2026-08-28

本文范围。本文对应仓库中的实际实现：在 OGBench Cube 上，从原始图像和随机初始化开始，联合训练 LeWM latent world model 与 boundary-anchored MC Goal Tail。本文列出训练 target、损失函数、方法差异和当前实验边界；不把尚未验证的结果扩展为通用结论。

## 1 一句话概括

Aligned E2E MC-GT-LeWM 是一个 **LeWM latent world model + goal-conditioned long-horizon cost** 的联合训练方法：它从 Cube 原始图像和随机初始化开始，同时学习短期 latent dynamics 与一个基于 Monte-Carlo future targets 的目标尾部代价（MC Goal Tail），再把两者接入同一个 CEM planner。

## 2 名称含义

| 名称             | 含义                                                                                        |
|----------------|-------------------------------------------------------------------------------------------|
| <b>LeWM</b>    | latent world model。编码观测和动作，预测未来 latent。                                                   |
| <b>MC-GT</b>   | Monte-Carlo Goal Tail。用离线轨迹中的未来 latent 计算到目标的折扣累计代价，不做 TD bootstrap。                      |
| <b>E2E</b>     | end-to-end。从原始图像随机初始化联合训练 world model 和 tail value，不使用冻结的 LeWM checkpoint 或 latent cache。 |
| <b>Aligned</b> | tail 使用与当前 LeWM 对齐的 latent 坐标、EMA target world model 和结构化零边界，避免把不同表示空间的 value 直接混用。       |

## 3 为什么需要 tail value

普通 LeWM CEM 通常主要依据候选动作 rollout 的终点 latent 与目标 latent 的距离。对于较长的 start-goal 间隔，这个终点距离可能过于短视，且对中间轨迹质量不敏感。

本方法让一个 goal-conditioned value head 估计：

给定当前 latent 历史和目标 latent，沿离线数据行为继续走一段时间的累计 latent goal cost。

因此 planner 的 cost 不只看一次终点距离，也可以利用训练过的长期目标代价。

## 4 模型结构

### 4.1 LeWM world model

在线 world model 从原始 Cube 图像得到 latent，并基于历史 latent 和动作预测未来 latent。它保留原 LeWM 的两项训练目标：

```
L_world = next-latent prediction MSE + 0.09 * SIGReg
```

正式配置使用 3 个 history frames、5-step model rollout、latent embedding size 192。

### 4.2 Boundary-anchored MC Goal Tail

tail value 接收：

```
V(history, goal) -> scalar cost
```

其中 history 包含预测 rollout 末端的 latent history 和此前动作, goal 是未来 latent。它不是普通的自由输出 MLP, 而是通过共享 scalar potential 构造:

$$V(h, g) = [\phi(h, g) - \phi(h, z_{\text{current}})]^2$$

所以它有两个重要性质:

1.  $V \geq 0$ ;
2.  $V(h, z_{\text{current}}) = 0$  是结构上严格成立的边界条件, 而不是依赖额外 penalty 学出来的近似值。

### 4.3 Monte-Carlo targets

对每个 tail 样本, 先让在线 LeWM 对前 5 个动作进行可微 rollout, 得到预测的 terminal history。然后用 EMA world model 编码真实离线序列的后续 latent, 构造未来 offset 1–16 的目标。

对 offset  $k$ , target 是从未来第 1 步到第  $k$  步的折扣 latent goal distance:

$$y_k = (1 - \gamma) * \sum_{j=1..k} \gamma^{j-1} * d(z_{T+j}, z_{T+k})$$

当前  $\gamma = 0.95$ , 目标 offset 在 1–16 间均匀采样, continuation policy 是离线数据中的 behavior continuation。该 target 是直接的 supervised MC target, 不使用 value bootstrap。

## 5 具体损失函数

下面把代码中真正计算的 loss 写成公式。令:

- $d = 192$  为 latent dimension;
- $H = 3$  为 latent history 长度;
- $R = 5$  为在线 LeWM rollout horizon;
- $K = 16$  为 tail 的最大 goal offset;
- $d_{\text{lat}}(u, v) = \frac{1}{d} \|u - v\|_2^2$  为 latent 平均平方距离。

### 5.1 LeWM 的短期 prediction loss

对 world view 中的每个短 clip, 在线 LeWM 编码得到真实 latent  $z$ , 并根据历史和动作得到预测 latent  $\hat{z}$ 。代码使用普通均方误差:

$$\mathcal{L}_{\text{pred}} = \text{mean} [(\hat{z} - z)^2].$$

同时保留 stable-worldmodel 中的 SIGReg 表示正则:

$$\mathcal{L}_{\text{world}} = \mathcal{L}_{\text{pred}} + 0.09 \mathcal{L}_{\text{SIGReg}}(z).$$

这里的 0.09、17 个 knots、1024 个 projections 和 effective batch size 128 都来自正式配置; SIGReg 的具体投影计算由 stable-worldmodel==0.1.1 的公开 API 完成。

## 5.2 MC tail target 的计算

对 tail view, 先用在线 LeWM 对前  $R = 5$  个动作做可微 rollout, 得到预测的末端 history  $\hat{h}_T$ 。另外, EMA target world model 编码同一条离线序列, 得到:

$$\bar{z}_{T+1}, \dots, \bar{z}_{T+K}.$$

第  $k$  个未来 goal 取为:

$$g_k = \bar{z}_{T+k}, \quad k = 1, \dots, K.$$

对应的直接 Monte-Carlo target 是:

$$y_k = (1 - \gamma) \sum_{j=1}^k \gamma^{j-1} d_{\text{lat}}(\bar{z}_{T+j}, \bar{z}_{T+k}), \quad \gamma = 0.95.$$

因此  $y_1 = 0$ , 而更远的 goal 会累积更多中间 future-latent 与最终 goal 的距离。这里没有 reward、TD bootstrap 或 beyond-clip 估计; 所有  $\bar{z}$  和  $y_k$  都是 stop-gradient 的 target。

## 5.3 Boundary-anchored value 的计算

tail head 先用一个共享的 scalar potential  $s_\psi(h, g)$ , 再构造 value/cost:

$$V_\psi(h, g) = [s_\psi(h, g) - s_\psi(h, z_{\text{cur}})]^2.$$

其中  $z_{\text{cur}}$  是 history 中最后一个 latent。因此:

$$V_\psi(h, g) \geq 0, \quad V_\psi(h, z_{\text{cur}}) = 0.$$

在网络结构上严格成立。它不是给普通 MLP 再加一个软 boundary penalty。

## 5.4 MC tail regression loss

对 batch 中的每个样本  $b$  和全部  $K = 16$  个 future offsets, 代码计算:

$$\mathcal{L}_{\text{tail}} = \frac{1}{BK} \sum_{b=1}^B \sum_{k=1}^K \left[ V_\psi \left( \text{GradScale}_{0.1}(\hat{h}_{T,b}), g_{b,k} \right) - y_{b,k} \right]^2.$$

这里的  $\text{GradScale}_{0.1}$  只把 tail loss 传回 online world model rollout 的梯度缩放为 0.1; tail value head 的参数梯度不缩放。也就是说, 同一个 tail loss 同时训练 value head, 并以较小梯度校准 LeWM 的长期 rollout。

## 5.5 最终联合 loss

训练时先计算 world loss，再计算 tail loss，最后联合为：

$$\mathcal{L}_{\text{joint}} = \mathcal{L}_{\text{world}} + \lambda_{\text{tail}} w(t) \mathcal{L}_{\text{tail}}$$

当前配置中：

$$\lambda_{\text{tail}} = 1, \quad w(t) = \min \left( 1, \frac{t+1}{0.05 T_{\text{train}}} \right).$$

即 tail loss 在训练前 5% 的 optimizer steps 内线性 warm up，之后权重为 1。两项 loss 在同一个 optimizer update 中更新 online LeWM 和 value head；随后用：

$$\bar{\theta} \leftarrow 0.995 \bar{\theta} + 0.005 \theta$$

更新 EMA target world model。梯度裁剪是优化步骤，不属于 loss 定义。

## 6 相关方法对比

下面只列与当前 MC-GT-LeWM 直接相关的四个方法：

| 方法                            | World model / training        | loss / planner                                                                    | 核心区别                                            |
|-------------------------------|-------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------|
| <b>Original LeWM</b>          | 训练 LeWM                       | $\mathcal{L}_{\text{world}}$ ; terminal latent-goal distance                      | 没有长期 tail，只有短期 latent prediction，作为 baseline。   |
| <b>Frozen MC-GT-LeWM</b>      | 加载并冻结已训练 LeWM，使用 latent cache | $\text{MSE}(V_{\psi}, y)$ ; LeWM terminal cost + MC tail                          | 只训练 value head，不会反过来改变 world model。             |
| <b>E2E Joint TD-GT-LeWM</b>   | 从原始图像随机初始化，联合训练               | $\text{MSE}(V_{\psi}, c_{t+1} + \gamma V_{\bar{\psi}})$ ; terminal cost + TD tail | 使用 TD bootstrap；旧实现没有结构化 exact-zero boundary。   |
| <b>Aligned E2E MC-GT-LeWM</b> | 从原始图像随机初始化，联合训练               | $\mathcal{L}_{\text{joint}}$ ; terminal cost + anchored MC tail                   | 直接 MC 全 offset、EMA target、anchored value，并联合更新。 |

表中 TD 行的即时 cost 是：

$$c_{t+1} = (1 - \gamma) d_{\text{lat}}(z_{t+1}, g),$$

并在已经到达 goal 的 offset 上关闭 bootstrap：

$$y_{TD} = c_{t+1} + \gamma \mathbf{1}[\Delta > 1] V_{\bar{\psi}}(h_{t+1}, g).$$

## 6.1 这些差异意味着什么

1. **Frozen vs E2E**: Frozen 版的 value 只能适应既有 latent; E2E 版的长期监督可以改变 latent dynamics, 使 representation 也考虑长期规划。
2. **TD vs MC**: TD 用下一时刻 value bootstrap, 误差会递推; 当前方法直接用离线轨迹中的 1-16 步 future latent 计算 target, 代价是需要更长序列和更多编码计算。
3. **普通 value vs anchored value**: 普通 MLP 不保证到达当前 goal 时 cost 为 0; 当前的平方 potential 构造从数学上保证这个边界, 同时保证 cost 非负。
4. **真实 history vs predicted history**: 当前 tail 不只在真实 latent history 上拟合, 而是输入在线 LeWM 5-step rollout 的 predicted terminal history, 因此训练路径和测试时 planner 的 imagined rollout 更一致。
5. 推理时 **tail** 不等于训练收益: tail 会改变 CEM candidate ranking, 但当前 300 个 paired episodes 中, 主要的稳定提升来自 Aligned world-model training; inference-time tail 本身没有显示稳定的 success-rate 增益。

## 7 Aligned 具体对齐了什么

它不是简单地在已经训练好的 LeWM 后面外挂一个 value head, 而是同时处理以下对齐问题:

- 表示对齐: target latent 来自在线 LeWM 的 EMA 副本, 和当前模型保持同一表示演化轨迹;
- 训练路径对齐: tail 的输入是 online LeWM 预测 rollout 得到的 terminal history, 而不是只在真实 latent 上训练;
- 目标边界对齐: 当前状态作为自己的 goal 时, tail cost 精确为零;
- 数据视图对齐: 同一次 optimizer update 使用 128 个独立短 clip 训练原始 LeWM loss, 另用 16 个 long clip 训练 MC tail, 避免用同一个短 clip 假装提供长期监督。

EMA target world model 的 decay 为 0.995。tail loss 在训练开始时 warm up, 并把传回 world model rollout 的梯度缩放为 0.1; value head 本身仍正常更新。

联合目标可以简写为:

```
L_joint = L_world + warmup(t) * L_MC_tail
```

两项 loss 在同一个 optimizer update 中更新同一个 online LeWM; EMA model 只作为无梯度 target encoder。

## 8 规划时如何使用

CEM solver 和 LeWM 的基础设置保持不变。候选动作 rollout 后, 使用:

```
planning cost = terminal latent-goal distance
                + boundary-anchored MC tail cost
```

当前 Cube 正式协议为 horizon 5、300 candidates、30 iterations、30 elites，每 5 个环境 step 重规划。

## 9 当前实验结果与证据边界

在固定 training seed 3072 的 Cube 评测中：

- 历史单次 planning selection (50 episodes):  $31/50 = 62\%$ ;
- 后续 6 组 matched selections (共 300 episodes)，完整方法 (Aligned world + anchored tail):  $167/300 = 55.67\%$ ;
- 同一批次的 Original LeWM world-only:  $153/300 = 51.0\%$ ;
- 去掉 inference-time tail 的 Aligned world-only:  $168/300 = 56.0\%$ 。

这些结果只有一个 training seed。当前证据更支持“长期监督改善了训练出的 world model”，而不是“推理时追加 tail 一定带来稳定收益”，暂不能据此声称统计意义上的方法优越性。

## 10 仓库中的实现入口

- 方法配置: `configs/methods/aligned_e2e_mc_gt_lewm.yaml`
- 训练配置: `configs/experiment/aligned_e2e_mc_gt_lewm_cube_train.yaml`
- 训练实现: `src/tdwm/training/aligned_e2e_mc_gt_lewm.py`
- tail value 实现: `src/tdwm/methods/goal_tail_value.py`
- 正式结果: `reports/aligned_e2e_mc_gt_lewm_cube_seed3072.md`
- 六组 matched 归档: `reports/aligned_acd_cube_o50_seed3072_planning_seeds42_47.md`

实现固定使用 `stable-worldmodel[all]==0.1.1`。